

Types overview

June 17, 2026 2:43 PM

<https://kotlinlang.org/docs/types-overview.html>

Types overview

 [Edit page](#) Last modified: 01 April 2025

In Kotlin, everything is an object in the sense that you can call member functions and properties on any variable. While certain types have an optimized internal representation as primitive values at runtime (such as numbers, characters, and booleans), they appear and behave like regular classes to you.

This section describes the basic types used in Kotlin:

- [Numbers and their unsigned counterparts](#)
- [Booleans](#)
- [Characters](#)
- [Strings](#)
- [Arrays](#)

To learn about other Kotlin types, such as `Nothing`, `Any`, and `Unit`, look through the Kotlin API reference:

- [Any](#) ↗
- [Nothing](#) ↗
- [Unit](#) ↗



[Learn how to perform type checks and casts in Kotlin.](#)

Numbers

 Edit page Last modified: 18 May 2026

The Kotlin number types represent:

- Integer values ([Byte ↗](#), [Short ↗](#), [Int ↗](#), and [Long ↗](#))
- Floating-point values ([Float ↗](#) and [Double ↗](#))

Use number types to store and process numeric data, for example, in arithmetic, counters, measurements, and other calculations.

Choose a number type

In most cases, you can refer to the following rules to determine the correct number type for your task:

- Use `Int` for whole numbers.
- Use `Long` for whole numbers outside the `Int` range.
- Use `Double` for decimal numbers.
- Use `Float` when lower precision is acceptable or required.
- Use `Byte` and `Short` when an API or data format requires them.

 Kotlin also provides [Unsigned integer types](#) as a Beta feature.

Integer types

Kotlin provides four integer types with different sizes and value ranges:

Type	Size (bits)	Min value	Max value
<code>Byte</code>	8	-128	127
<code>Short</code>	16	-32768	32767
<code>Int</code>	32	-2,147,483,648 (-2^{31})	2,147,483,647 ($2^{31} - 1$)
<code>Long</code>	64	-9,223,372,036,854,775,808 (-2^{63})	9,223,372,036,854,775,807 ($2^{63} - 1$)

Numbers

Choose a number type

Integer types

Declare integer values

Floating-point types

Declare floating-point values

Arithmetic operations

Integer division

Type conversion

Mixed numeric expressions

Data overflow

Overflow in operations

Overflow in negation

Narrowing conversions

Bitwise operations

Floating-point number comparison

Boxing and caching numbers on the JVM

Declare integer values [↗](#)

Kotlin supports the following literal forms for integer values:

- Decimals: `123`
- Hexadecimals: `0x0F`
- Binaries: `0b00001011`

 Kotlin does not support octal literals.

To declare a numeric value, specify the type explicitly:

```
val one: Int = 1

// Use underscores to improve readability
val oneBillion: Long = 1_000_000_000
val hexBytes: Int = 0x7F_EC_DE_5E
val bytes: Int = 0b01010010_01101001_10010100_10010010

val oneByte: Byte = 1
val oneShort: Short = 1
```

You can also append the `L` suffix, to declare a `Long` value:

```
val oneLong = 1L
```

When you declare a numeric type explicitly, the compiler checks that the value fits in the range of that type:

```
// Value fits in Byte
val oneByte: Byte = 1

// Error: the value does not fit in Byte
val tooBig: Byte = 128
```

When you do not specify a numeric type, Kotlin infers `Int` if the value fits in the `Int` range. Otherwise, Kotlin infers `Long`:

```
val million = 1_000_000 // Int
val threeBillion = 3_000_000_000 // Long
```

If a value can be absent, use nullable types:

```
val maybeAbsent: Int? = null
```

Floating-point types

For numbers with a fractional part, Kotlin provides `Float` and `Double`.

Floating-point types follow the IEEE 754 standard [↗](#). `Float` reflects the **single precision**. `Double` reflects the **double precision**.

Floating-point types

For numbers with a fractional part, Kotlin provides `Float` and `Double`.

Floating-point types follow the IEEE 754 standard [↗]. `Float` reflects the **single precision**. `Double` reflects the **double precision**.

Floating-point types differ in size and precision:

Type	Size (bits)	Significant bits	Exponent bits	Decimal digits
<code>Float</code>	32	24	8	6-7
<code>Double</code>	64	53	11	15-16

Declare floating-point values

To declare a floating-point literal, include a decimal point (`.`) or use exponent notation:

```
val pi = 3.14
val avogadro = 6.02214076e23
```

By default, Kotlin infers floating-point literals as `Double`. To declare a `Float`, add the `f` or `F` suffix:

```
val pi = 3.14 // Double
val eFloat = 2.7182817f // Float
```

 Kotlin rounds a `Float` literal that contains more precision than `Float` can store.

If a value can be absent, use nullable types:

```
val maybeAbsent: Double? = null
```

Arithmetic operations

Kotlin supports the standard arithmetic operations on numbers: `+`, `-`, `*`, `/`, and `%`.

Use these operators to perform common calculations:

+

```
println(1 + 2) // 3
println(2_500_000_000L - 1L) // 2499999999
println(3.14 * 2.71) // 8.5094
println(10.0 / 3) // 3.3333333333333335
```

[Open in Playground](#) →

Target: JVM Running on v.24.0

The result type depends on the types of the operands. Learn more in [Mixed numeric expressions](#).

The result type depends on the types of the operands. Learn more in [Mixed numeric expressions](#).



You can override these operators in custom number classes. For more information, see [Operator overloading](#).

Integer division

Division between integer values always returns an integer result. The compiler discards the fractional part:

```
val intValue = 5 / 2
println(intValue) // 2

val longValue = 5L / 2
println(longValue) // 2
```

[Open in Playground](#) →

Target: JVM Running on v.24.0

To return a floating-point result, make at least one operand a `Float` or `Double`:

```
val a = 5 / 2.0
println(a) // 2.5

val b = 5 / 2.toDouble()
println(b) // 2.5
```

[Open in Playground](#) →

Target: JVM Running on v.24.0

Type conversion

Numeric types are not subtypes of one another. Kotlin requires explicit conversions to avoid silent data loss and unexpected behavior.

For example, a function that expects `Double` cannot accept an `Int` or a `Float` value without conversion:

```
fun printDouble(x: Double) {
    print(x)
}

val x = 1.0
val xInt = 1
val xFloat = 1.0f
val one: Double = 1 // Error: initializer type mismatch

printDouble(x) // OK
printDouble(xInt) // Error: argument type mismatch
printDouble(xFloat) // Error: argument type mismatch
```

[Open in Playground](#) →

Target: JVM Running on v.24.0

All number types support conversions to other number types. To convert a number to another type, use an explicit conversion function:

All number types support conversions to other number types. To convert a number to another type, use an explicit conversion function:

- `toByte()`
- `toShort()`
- `toInt()`
- `toLong()`
- `toFloat()`
- `toDouble()`

For example, the following code converts an `Int` value to `Double` :

```
val intValue: Int = 1
val doubleValue = intValue.toDouble()

println(doubleValue) // 1.0
```

Open in Playground → Target: JVM Running on v2.4.0

When you convert a floating-point value to an integer type, the compiler discards the fractional part:

```
val d: Double = 1.5
val l: Long = d.toLong()

println(l) // 1
```

Open in Playground → Target: JVM Running on v2.4.0

Mixed numeric expressions

Kotlin does not support implicit conversion for assignments or function arguments. However, you can combine different numeric types in arithmetic expressions. In such cases, Kotlin determines a result type based on the operand types, and arithmetic operators handle the conversion automatically:

```
val intNumber: Int = 1
val longNumber: Long = 1000
val result = intNumber + longNumber // 1001, Long
```

If you try to assign the result to a smaller type, the compiler reports an error:

```
val intNumber: Int = 1
val longNumber: Long = 1000
val result: Int = intNumber + longNumber
// Error: Initializer type mismatch
```

Data overflow

Data overflow

Numeric types can represent only values within their defined ranges.

If the result of an operation falls outside that range, overflow occurs. If you convert a value to a smaller numeric type, the converted value may not preserve the original numeric value.

This behavior can affect the result of your code even when the compiler accepts it.

Overflow in operations

Each integer type can store only values within its defined range. When the result of an arithmetic operation exceeds that range, **data overflow** occurs:

+

```
val intNumber: Int = 2147483647
// Max Int value is 2147483647
println(intNumber + 1) // -2147483648
```

Open in Playground →

Target: JVM Running on v2.4.0

Here, the result wraps around because the value no longer fits in `Int`.

 The compiler does not automatically produce an error when integer overflow occurs.

Overflow in negation

Overflow can also occur during negation. For example, you cannot represent the positive counterpart of `Int.MIN_VALUE` as an `Int`.

+

```
val min = Int.MIN_VALUE
println(-min) // -2147483648
```

Open in Playground →

Target: JVM Running on v2.4.0

Narrowing conversions

When you convert a value to a smaller integer type, the result may not preserve the original numeric value:

+

```
val large: Int = 130
val narrowed: Byte = large.toByte()

println(narrowed) // -126
```

Open in Playground →

Target: JVM Running on v2.4.0

However, since floating-point types follow the IEEE 754 Standard ², very large results can become `Infinity`:

+

```
println(Double.MAX_VALUE * 2) // Infinity
```

Open in Playground →

Target: JVM Running on v2.4.0

Open in Playground →

Target: JVM Running on v2.4.0

Bitwise operations

Kotlin provides **bitwise operations** for `Int` and `Long`. These operations are represented by a set of [infix functions](#) and `inv()`.

```
val x = 1

println(x shl 2) // 4
println(x and 0x000F000) // 0
```

Open in Playground →

Target: JVM Running on v2.4.0

Bitwise operations include:

- `shl()` – signed shift left
- `shr()` – signed shift right
- `ushr()` – unsigned shift right
- `and()` – bitwise AND
- `or()` – bitwise OR
- `xor()` – bitwise XOR
- `inv()` – bitwise inversion

Floating-point number comparison

In Kotlin, floating-point comparison depends on the static type of the operands.

When the operands are statically known to be `Float` or `Double`, operations on the numbers and the range that they form follow the [IEEE 754 Standard for Floating-Point Arithmetic](#).

However, in generic use cases (such as `Any`, `Comparable<...>`, or `Collection<T>`), behavior differs for operands that are not statically typed as floating-point numbers. In these cases, Kotlin uses the `equals()` and `compareTo()` implementations for `Float` and `Double`.

As a result:

- `NaN` is considered equal to itself
- `NaN` is considered greater than any other element including `POSITIVE_INFINITY`
- `-0.0` is considered less than `0.0`

The following example shows the difference between operands statically typed as floating-point numbers and operands used through generic types:


```

fun generalizedEquals(a: Any, b: Any): Boolean {
    return a == b
}

fun main() {
    // Operands statically typed as floating-point numbers
    println(Double.NaN == Double.NaN) // false
    println(0.0 == -0.0) // true

    // Operands used through a non-floating-point static type
    println(generalizedEquals(Double.NaN, Double.NaN)) // true
    println(generalizedEquals(0.0, -0.0)) // false
}

```

[Open in Playground →](#)

Target: JVM Running on v2.4.0

Boxing and caching numbers on the JVM [↗](#)

On the JVM, non-nullable numeric values are usually stored using primitive types, such as `int`, `long`, or `double`. However, when you use generic types or nullable numeric types like `Int?`, the value is boxed and represented as an object.

The JVM applies a memory optimization technique [↗](#) to small numbers by caching their boxed representations. As a result, boxed numbers with the same value can be referentially equal.

For example, the JVM caches boxed `Integer` values in the range `-128` to `127`. Therefore, the following code returns `true`:

```

val score: Int = 100
val savedScore: Int? = score
val displayedScore: Int? = score

println(savedScore === displayedScore) // true

```

[Open in Playground →](#)

Target: JVM Running on v2.4.0

For values outside the cached range, boxed values are separate objects. In that case, they are not referentially equal, even if their values are structurally equal. For this reason, use `==` to compare numeric values:

```

val score: Int = 10000
val savedScore: Int? = score
val displayedScore: Int? = score

println(savedScore === displayedScore) // false
println(savedScore == displayedScore) // true

```

[Open in Playground →](#)

Target: JVM Running on v2.4.0

Booleans

 [Edit page](#) Last modified: 05 May 2026

The `Boolean` type represents logical values: `true` and `false`.

Use `Boolean` values in functions that answer yes-or-no questions, and in the `while`, `if`, and `when` conditions.

Declare a `Boolean` variable

To declare a `Boolean` variable, assign it `true` or `false`.

You can specify the `Boolean` type explicitly or let Kotlin infer it from the value:

```
val isTrue: Boolean = true
val isFalse = false // Kotlin infers Boolean
```

If a value can be `null`, use `Boolean?`:

```
val isEnabled: Boolean? = null
```

 You cannot assign an integer value to a `Boolean` variable. In Kotlin, `0` and `1` are not `Boolean` values.

Produce `Boolean` values

You can use comparison expressions and functions to produce `Boolean` values:

```
val number = 10
val isPositive = number > 0
println(isPositive) // true

val language = "Kotlin"
val isEmpty = language.isEmpty()
println(isEmpty) // false
```

[Open in Playground](#) →

Target: JVM Running on v2.4.0

You can use the results in conditions and other expressions as well:

```
val number = 10
val isPositive = number > 0 // true

if (isPositive) {
```

Booleans

Declare a `Boolean` variable

Produce `Boolean` values

`Boolean` operations

Operator precedence

`Boolean` in conditions

```
val isPositive = number > 0 // true

if (isPositive) {
    println("The number is positive.")
}
```

[Open in Playground →](#)

Target: JVM Running on v2.4.0

Boolean operations

Kotlin provides operators and infix functions for working with `Boolean` values. You can use them to invert a `Boolean` value or combine multiple `Boolean` values into a single result.

Negation (NOT)

The NOT operator inverts a `Boolean` value.

To use NOT, place the `!` operator before a `Boolean` value:

```
val isOn = true
val isOff = !isOn // isOff is false
```

Logical AND

The AND operator returns `true` only if both operands are `true`.

To use logical AND, place the `&&` operator between operands:

```
val a = false && false // false
val b = false && true  // false
val c = true  && false  // false
val d = true  && true   // true
```

 If the first operand is `false`, the `&&` operator skips the second operand. To evaluate both operands, use the `and` infix function instead.

Logical OR

The OR operator returns `true` if at least one operand is `true`.

To use logical OR, place the `||` operator between operands:

```
val a = false || false // false
val b = false || true  // true
val c = true  || false  // true
val d = true  || true   // true
```

 If the first operand is `true`, the `||` operator skips the second operand. To evaluate both operands, use the `or` infix function instead.

i If the first operand is `true`, the `||` operator skips the second operand. To evaluate both operands, use the `or` infix function instead.

Exclusive OR (XOR)

The exclusive OR (XOR) operation returns `true` if the operands have different values.

To use XOR, write `xor` between operands:

```
val a = false xor false // false
val b = false xor true  // true
val c = true  xor false  // true
val d = true  xor true   // false
```

i `xor` is an infix function, not an operator.
Learn more about `Boolean` functions in the [API Reference](#).

Operator precedence

If an expression contains multiple logical operations and no parentheses to specify the evaluation order, Kotlin applies precedence rules. Operations with higher precedence are evaluated before operations with lower precedence.

For the `Boolean` operations described in this section, the precedence order is as follows:

1. `!`
2. `xor` (and other infix functions)
3. `&&`
4. `||`

In the following example, the compiler evaluates `&&` before `||`:

+

```
val result = true || false && false
println(result) // true
```

[Open in Playground](#) →

Target: JVM Running on v.24.0

To make evaluation order explicit, use parentheses:

+

```
val result = (true || false) && false
println(result) // false
```

[Open in Playground](#) →

Target: JVM Running on v.24.0

Boolean in conditions

Boolean in conditions

`if`, `when`, and `while` evaluate `Boolean` expressions to direct program flow.

if expressions

+

```
val number = 4
val isEven = number % 2 == 0

// Condition already has the `Boolean` type
// You do not need to compare it to `true` or `false`
if (isEven) {
    println("The number is even.")
} else {
    println("The number is odd.")
}
```

Open in Playground → Target: JVM Running on v2.4.0

when expressions

+

```
val number = 3

when {
    number > 0 -> println("The number is positive.")
    number < 0 -> println("The number is negative.")
    else -> println("The number is zero.")
}
```

Open in Playground → Target: JVM Running on v2.4.0

while loops

+

```
var isCalculating = true

while (isCalculating) {
    println("Calculating...")
    isCalculating = false
}
```

Open in Playground → Target: JVM Running on v2.4.0

Characters

 Edit page Last modified: 04 May 2026

The `Char`  type represents a single character as a UTF-16 code unit.

Use `Char` for individual character values, such as letters, digits, punctuation marks, or whitespace. For sequences of characters, use `String`.

Characters
Syntax
Nullable values
Unicode support
Basic Multilingual Plane
Supplementary characters
Escape sequences
Operations

whitespace. For sequences of characters, use `String`.



`Char` is not a numeric type, but each character has a numeric Unicode value that you can access. See [Character conversion](#).

Escape sequences

Operations

Character comparison

Character processing

Character arithmetic

Character conversion

Syntax

To declare a character, enclose the value in single quotes (`' '`). You can specify the `Char` type explicitly or let Kotlin infer it from the value:

```
val letter: Char = 'a'

// Kotlin infers Char because the values are written in single quotes
val digit = '1'
val symbol = '!'
val space = ' '
val separator = ':'
```

A character literal must contain exactly one character. Otherwise, the Kotlin compiler reports an error:

```
val invalid = 'AB' // Error
val invalidEmpty = '' // Error
```

Nullable values

To store a nullable value, use `Char?` :

```
val maybeAbsent: Char? = null
```



On the JVM, nullable `Char` values are boxed when needed. The same applies to numeric types.

Unicode support

Kotlin represents `Char` values as UTF-16 code units. This means that a single `Char` stores one UTF-16 code unit, not necessarily one complete Unicode character.

Basic Multilingual Plane

A single `Char` can store values in the range from `\u0000` to `\uFFFF` . This range covers the Basic Multilingual Plane (BMP) that includes characters for almost all modern languages and a large number of symbols.

To specify a character by the Unicode value, use `\u` followed by four-digit hexadecimal value from the [Unicode table](#) ↗:

```
val unicodeNumber = '\u0031' // Equals '1'
```



```
val unicodeNumber = '\u0031' // Equals '1'
```

Supplementary characters

Unicode characters outside the BMP, such as emojis and some historic scripts, cannot be represented by a single `Char`. In UTF-16, they are encoded as a **surrogate pair**, where two `Char` values together represent one Unicode character in a `String`:

```
fun main() {
    //sampleStart
    val emoji = "👋"

    println(emoji.length) // 2
    println(emoji[0])      // First surrogate
    println(emoji[1])      // Second surrogate
    //sampleEnd
}
```



To handle 32-bit symbols individually, use Unicode code points stored as `Int` values.

Escape sequences

Use escape sequences for special characters that are difficult to write directly in source code or have a special meaning.

Every escape sequence begins with a backslash (`\`).

Supported sequence	Description
<code>\t</code>	Tab
<code>\b</code>	Backspace
<code>\n</code>	New Line (LF)
<code>\r</code>	Carriage Return (CR)
<code>\'</code>	Single quotation mark
<code>\"</code>	Double quotation mark
<code>\\</code>	Backslash
<code>\\$</code>	Dollar sign

For example:

```
val newLine = '\n'
```

```
val newline = '\n'
val dollar = '\$'
val backslash = '\\'
```

Operations

`Char` supports comparison, inspection, case conversion, and explicit numeric conversion.

Character comparison

To compare `Char` values, use standard comparison operators such as `==`, `!=`, `<`, `>`, `<=`, and `>=`.

Kotlin compares `Char` values by their numeric Unicode values and returns a `Boolean` value:

```
val before = 'a' < 'b' // true
val after = 'c' > 'd' // false
val different = 'A' == 'a' // false
val equal = 'A' == 'A' // true
```

Character processing

Kotlin provides functions for inspection and case conversion of character values. For example:

```
val myChar = 'A'
// Checks if the character represents a digit
println(myChar.isDigit()) // false
// Checks if the character represents an uppercase letter
println(myChar.isUpperCase()) // true
// Returns a lowercase version
println(myChar.lowercaseChar()) // 'a'
```

[Open in Playground →](#)

Target: JVM Running on v2.4.0

 Learn more about available functions in the [API Reference ↗](#).

Character arithmetic

You can create another character value by adding or subtracting an integer:

```
val a = 'a'

println(a + 1) // b
println(a + 2) // c
println(a - 32) // A
```

[Open in Playground →](#)

Target: JVM Running on v2.4.0

 These operations follow Unicode values, not language-specific alphabet rules.

 These operations follow Unicode values, not language-specific alphabet rules.

You can also use the increment (`++`) and decrement (`--`) operators in the prefix and postfix forms with mutable variables:

+

```
var a = 'A'

a += 10
println(a) // 'K'

println(++a) // 'L' prefix increment
println(a++) // 'L' postfix increment
println(a) // 'M'

println(--a) // 'L' prefix decrement
println(a--) // 'L' postfix decrement
println(a) // 'K'
```

[Open in Playground →](#)

Target: JVM Running on v2.4.0

Character conversion

To convert `Char` to a numeric type, use explicit conversion:

- Use `.code` to get the numeric Unicode value of a character:

+

```
val letter = 'A'
println(letter.code) // 65
```

[Open in Playground →](#)

Target: JVM Running on v2.4.0

- If a character represents a decimal digit, use `digitToInt()` :

+

```
val digit = '7'
println(digit.digitToInt()) // 7
```

[Open in Playground →](#)

Target: JVM Running on v2.4.0

 If the character may not be a valid digit, use `digitToIntOrNull()` .

Strings

 [Edit page](#) Last modified: 04 February 2026

Strings in Kotlin are represented by the type `String` [↗](#).

 On the JVM, an object of `String` type in UTF-16 encoding uses approximately 2 bytes per character.

Generally, a string value is a sequence of characters in double quotes (`"`):

— [Strings](#)

— [String literals](#)

— [Escaped strings](#)

— [Multiline strings](#)

— [String templates](#)

— [Multi-dollar string interpolation](#)

— [String formatting](#)

Generally, a string value is a sequence of characters in double quotes ("):

```
val str = "abcd 123"
```

Elements of a string are characters that you can access via the indexing operation: `s[i]`. You can iterate over these characters with a `for` loop:

```
for (c in str) {  
    println(c)  
}
```

[Open in Playground →](#)

Target: JVM Running on v2.4.0

Strings are immutable. Once you initialize a string, you can't change its value or assign a new value to it. All operations that transform strings return their results in a new `String` object, leaving the original string unchanged:

```
val str = "abcd"  
  
// Creates and prints a new String object  
println(str.uppercase())  
// ABCD  
  
// The original string remains the same  
println(str)  
// abcd
```

[Open in Playground →](#)

Target: JVM Running on v2.4.0

To concatenate strings, use the `+` operator. This also works for concatenating strings with values of other types, as long as the first element in the expression is a string:

```
val s = "abc" + 1  
println(s + "def")  
// abc1def
```

[Open in Playground →](#)

Target: JVM Running on v2.4.0

 In most cases using [string templates](#) or [multiline strings](#) is preferable to string concatenation.

String literals

Kotlin has two types of string literals:

- [Escaped strings](#)
- [Multiline strings](#)

Escaped strings

Escaped strings can contain escaped characters. Here's an example of an escaped string:

Escaped strings can contain escaped characters.
Here's an example of an escaped string:

```
val s = "Hello, world!\n"
```

Escaping is done in the conventional way, with a backslash (`\`).
See [Characters](#) page for the list of supported escape sequences.

Multiline strings

Multiline strings can contain newlines and arbitrary text. It is delimited by a triple quote (`"""`), contains no escaping and can contain newlines and any other characters:

```
val text = """
    for (c in "foo")
        print(c)
    """
```

To remove leading whitespace from multiline strings, use the `trimMargin()`  function:

```
val text = """
    |Tell me and I forget.
    |Teach me and I remember.
    |Involve me and I learn.
    |(Benjamin Franklin)
    """.trimMargin()
```

By default, a pipe symbol `|` is used as margin prefix, but you can choose another character and pass it as a parameter, like `trimMargin(">")`.

String templates

String literals may contain **template expressions** – pieces of code that are evaluated and whose results are concatenated into a string. When a template expression is processed, Kotlin automatically calls the `.toString()` function on the expression's result to convert it into a string. A template expression starts with a dollar sign (`$`) and consists of either a variable name:

```
val i = 10
println("i = $i")
// i = 10

val letters = listOf("a", "b", "c", "d", "e")
println("Letters: $letters")
// Letters: [a, b, c, d, e]
```

[Open in Playground](#) →

Target: JVM Running on v.2.4.0

or an expression in curly braces:

or an expression in curly braces:

```
val s = "abc"
println("${s.length} is ${s.length}")
// abc.length is 3
```

[Open in Playground](#) →

Target: JVM Running on v.2.4.0

You can use templates both in multiline and escaped strings. However, multiline strings don't support backslash escaping. To insert the dollar sign `$` in a multiline string before any symbol allowed at the beginning of an identifier `^`, use the following syntax:

```
val price = """
${'$'}_9.99
"""
```

 To avoid `${'$'}` sequences in strings, you can use the Experimental [multi-dollar string interpolation feature](#).

Multi-dollar string interpolation

Multi-dollar string interpolation allows you to specify how many consecutive dollar signs are required to trigger interpolation. Interpolation is the process of embedding variables or expressions directly into a string.

While you can escape literals for single-line strings, multiline strings in Kotlin don't support backslash escaping. To include dollar signs (`$`) as literal characters, you must use the `${'$'}` construct to prevent string interpolation. This approach can make code harder to read, especially when strings contain multiple dollar signs.

Multi-dollar string interpolation simplifies this by letting you treat dollar signs as literal characters in both single-line and multiline strings. For example:

```
val KClass<*>.jsonSchema : String
    get() = $$"""
    {
        "$schema": "https://json-schema.org/draft/2020-12/schema",
        "$id": "https://example.com/product.schema.json",
        "$dynamicAnchor": "meta",
        "title": "$${simpleName ?: qualifiedName ?: "unknown"}",
        "type": "object"
    }
    """
```

Here, the `$$` prefix specifies that two consecutive dollar signs are required to trigger string interpolation. Single dollar signs remain as literal characters.

You can adjust how many dollar signs trigger interpolation. For example, using three consecutive dollar signs (`$$$`) allows `$` and `$$` to remain as literals while enabling interpolation with `$$$` :

```
val productName = "carrot"
val requestedData =
```



```

val productName = "carrot"
val requestedData =
    $$$"""{
        "currency": "$",
        "enteredAmount": "42.45 $$",
        "$$serviceField": "none",
        "product": "$$$productName"
    }
    """"

println(requestedData)
//{
//    "currency": "$",
//    "enteredAmount": "42.45 $$",
//    "$$serviceField": "none",
//    "product": "carrot"
//}

```

Here, the `$$$` prefix allows the string to include `$` and `$$` without requiring the ``${}`` construct for escaping.

Multi-dollar string interpolation doesn't affect existing code that uses single-dollar string interpolation. You can continue using a single `$` as before and apply multi-dollar signs when you need to handle literal dollar signs in strings.

String formatting

 String formatting with the `String.format()` function is only available in Kotlin/JVM.

To format a string to your specific requirements, use the `String.format()` [function](#).

The `String.format()` function accepts a format string and one or more arguments. The format string contains one placeholder (indicated by `%`) for a given argument, followed by format specifiers. Format specifiers are formatting instructions for the respective argument, consisting of flags, width, precision, and conversion type. Collectively, format specifiers shape the output's formatting. Common format specifiers include `%d` for integers, `%f` for floating-point numbers, and `%s` for strings. You can also use the `argument_index$` syntax to reference the same argument multiple times within the format string in different formats.

 For a detailed understanding and an extensive list of format specifiers, see [Java's Class Formatter documentation](#).

Let's look at an example:

```

+
// Formats an integer, adding leading zeroes to reach a length of seven characters
val integerNumber = String.format("%07d", 31416)
println(integerNumber)
// 0031416

// Formats a floating-point number to display with a + sign and four decimal places
val floatNumber = String.format("%+.4f", 3.141592)
println(floatNumber)
// +3.1416

// Formats two strings to uppercase, each taking one placeholder

```

```
println(floatNumber)
// +3.1416

// Formats two strings to uppercase, each taking one placeholder
val helloString = String.format("%S %S", "hello", "world")
println(helloString)
// HELLO WORLD

// Formats a negative number to be enclosed in parentheses, then repeats the same
val negativeNumberInParentheses = String.format("(d means %1$d", -31416)
println(negativeNumberInParentheses)
// (31416) means -31416
```

[Open in Playground →](#)

Target: JVM Running on v.2.4.0

The `String.format()` function provides similar functionality to string templates. However, the `String.format()` function is more versatile because there are more formatting options available.

In addition, you can assign the format string from a variable. This can be useful when the format string changes, for example, in localization cases that depend on the user locale.

Be careful when using the `String.format()` function because it can be easy to mismatch the number or position of the arguments with their corresponding placeholders.

Arrays

[Edit page](#) Last modified: 01 April 2025

An array is a data structure that holds a fixed number of values of the same type or its subtypes. The most common type of array in Kotlin is the object-type array, represented by the `Array` class.

i If you use primitives in an object-type array, this has a performance impact because your primitives are boxed into objects. To avoid boxing overhead, use primitive-type arrays instead.

When to use arrays

Use arrays in Kotlin when you have specialized low-level requirements that you need to meet. For example, if you have performance requirements beyond what is needed for regular applications, or you need to build custom data structures. If you don't have these sorts of restrictions, use collections instead.

Collections have the following benefits compared to arrays:

- Collections can be read-only, which gives you more control and allows you to write robust code that has a clear intent.
- It is easy to add or remove elements from collections. In comparison, arrays are fixed in size. The only way to add or remove elements from an array is to create a new array each time, which is very inefficient:

```
+
var riversArray = arrayOf("Nile", "Amazon", "Yangtze")

// Using the += assignment operation creates a new riversArray,
// copies over the original elements and adds "Mississippi"
riversArray += "Mississippi"
```

- Arrays
- When to use arrays
- Create arrays
- Nested arrays
- Access and modify elements
- Work with arrays
- Pass variable number of arguments to a function
- Compare arrays
- Transform arrays
- Convert arrays to collections
- Primitive-type arrays
- What's next?

```
// Using the += assignment operation creates a new riversArray,
// copies over the original elements and adds "Mississippi"
riversArray += "Mississippi"
println(riversArray.joinToString())
// Nile, Amazon, Yangtze, Mississippi
```

[Open in Playground →](#)

Target: JVM Running on v24.0

- You can use the equality operator (==) to check if collections are structurally equal. You can't use this operator for arrays. Instead, you have to use a special function, which you can read more about in [Compare arrays](#).

For more information about collections, see [Collections overview](#).

Create arrays [↗](#)

To create arrays in Kotlin, you can use:

- functions, such as [arrayOf\(\)](#) [↗](#), [arrayOfNulls\(\)](#) [↗](#) or [emptyArray\(\)](#) [↗](#).
- the `Array` constructor.

This example uses the [arrayOf\(\)](#) [↗](#) function and passes item values to it:

```
+
// Creates an array with values [1, 2, 3]
val simpleArray = arrayOf(1, 2, 3)
println(simpleArray.joinToString())
// 1, 2, 3
```

[Open in Playground →](#)

Target: JVM Running on v24.0

This example uses the [arrayOfNulls\(\)](#) [↗](#) function to create an array of a given size filled with `null` elements:

```
+
// Creates an array with values [null, null, null]
val nullArray: Array<Int?> = arrayOfNulls(3)
println(nullArray.joinToString())
// null, null, null
```

[Open in Playground →](#)

Target: JVM Running on v24.0

This example uses the [emptyArray\(\)](#) [↗](#) function to create an empty array :

```
var exampleArray = emptyArray<String>()
```

- i** You can specify the type of the empty array on the left-hand or right-hand side of the assignment due to Kotlin's type inference.

For example:

```
var exampleArray = emptyArray<String>()

var exampleArray: Array<String> = emptyArray()
```

The `Array` constructor takes the array size and a function that returns values for array elements given its index:

```
// Creates an Array<Int> that initializes with zeros [0, 0, 0]
val initArray = Array<Int>(3) { 0 }
println(initArray.joinToString())
// 0, 0, 0

// Creates an Array<String> with values ["0", "1", "4", "9", "16"]
val asc = Array(5) { i -> (i * i).toString() }
asc.forEach { print(it) }
// 014916
```

[Open in Playground](#) →

Target: JVM Running on v.24.0

i Like in most programming languages, indices start from 0 in Kotlin.

Nested arrays

Arrays can be nested within each other to create multidimensional arrays:

```
// Creates a two-dimensional array
val twoDArray = Array(2) { Array<Int>(2) { 0 } }
println(twoDArray.contentDeepToString())
// [[0, 0], [0, 0]]

// Creates a three-dimensional array
val threeDArray = Array(3) { Array(3) { Array<Int>(3) { 0 } } }
println(threeDArray.contentDeepToString())
// [[[0, 0, 0], [0, 0, 0], [0, 0, 0]], [[0, 0, 0], [0, 0, 0], [0, 0, 0]], [[0, 0, 0], [0, 0, 0], [0, 0, 0]]]
```

[Open in Playground](#) →

Target: JVM Running on v.24.0

i Nested arrays don't have to be the same type or the same size.

Access and modify elements

Arrays are always mutable. To access and modify elements in an array, use the indexed access operator `[]` :

```
val simpleArray = arrayOf(1, 2, 3)
val twoDArray = Array(2) { Array<Int>(2) { 0 } }

// Accesses the element and modifies it
simpleArray[0] = 10
twoDArray[0][0] = 2

// Prints the modified element
println(simpleArray[0].toString()) // 10
println(twoDArray[0][0].toString()) // 2
```



```
// Prints the modified element
println(simpleArray[0].toString()) // 10
println(twoDArray[0][0].toString()) // 2
```

Open in Playground →

Target: JVM Running on v.2.4.0

Arrays in Kotlin are **invariant**. This means that Kotlin doesn't allow you to assign an `Array<String>` to an `Array<Any>` to prevent a possible runtime failure. Instead, you can use `Array<out Any>`. For more information, see [Type Projections](#).

Work with arrays

In Kotlin, you can work with arrays by using them to pass a variable number of arguments to a function or perform operations on the arrays themselves. For example, comparing arrays, transforming their contents or converting them to collections.

Pass variable number of arguments to a function

In Kotlin, you can pass a variable number of arguments to a function via the `vararg` parameter. This is useful when you don't know the number of arguments in advance, like when formatting a message or creating an SQL query.

To pass an array containing a variable number of arguments to a function, use the **spread** operator (`*`). The spread operator passes each element of the array as individual arguments to your chosen function:

```
fun main() {
    val lettersArray = arrayOf("c", "d")
    printAllStrings("a", "b", *lettersArray)
    // abcd
}

fun printAllStrings(vararg strings: String) {
    for (string in strings) {
        print(string)
    }
}
```

Open in Playground →

Target: JVM Running on v.2.4.0

For more information, see [Variable number of arguments \(varargs\)](#).

Compare arrays

To compare whether two arrays have the same elements in the same order, use the `.contentEquals()` and `.contentDeepEquals()` functions:

```
val simpleArray = arrayOf(1, 2, 3)
val anotherArray = arrayOf(1, 2, 3)

// Compares contents of arrays
println(simpleArray.contentEquals(anotherArray))
// true

// Using infix notation, compares contents of arrays after an element
// is changed
simpleArray[0] = 10
println(simpleArray contentEquals anotherArray)
// false
```

Open in Playground →

Target: JVM Running on v.2.4.0

```
// false
```

[Open in Playground](#) →

Target: JVM Running on v.2.4.0

⚠ Don't use equality (`==`) and inequality (`!=`) operators to compare the contents of arrays. These operators check whether the assigned variables point to the same object.

To learn more about why arrays in Kotlin behave this way, see our [blog post](#).

Transform arrays

Kotlin has many useful functions to transform arrays. This document highlights a few but this isn't an exhaustive list. For the full list of functions, see our [API reference](#).

Sum

To return the sum of all elements in an array, use the `.sum()` function:

```
val sumArray = arrayOf(1, 2, 3)

// Sums array elements
println(sumArray.sum())
// 6
```

[Open in Playground](#) →

Target: JVM Running on v.2.4.0

i The `.sum()` function can only be used with arrays of numeric data types, such as `Int`.

Shuffle

To randomly shuffle the elements in an array, use the `.shuffle()` function:

```
val simpleArray = arrayOf(1, 2, 3)

// Shuffles elements [3, 2, 1]
simpleArray.shuffle()
println(simpleArray.joinToString())

// Shuffles elements again [2, 3, 1]
simpleArray.shuffle()
println(simpleArray.joinToString())
```

[Open in Playground](#) →

Target: JVM Running on v.2.4.0

Convert arrays to collections

If you work with different APIs where some use arrays and some use collections, then you can convert your arrays to [collections](#) and vice versa.

Convert to List or Set

To convert an array to a `List` or `Set`, use the `.toList()` and `.toSet()` functions.

```
val simpleArray = arrayOf("a", "b", "c", "c")

// Converts to a Set
```



```
val simpleArray = arrayOf("a", "b", "c", "c")
```

```
// Converts to a Set
println(simpleArray.toSet())
// [a, b, c]

// Converts to a List
println(simpleArray.toList())
// [a, b, c, c]
```

[Open in Playground →](#)

Target: JVM Running on v.2.4.0

Convert to Map

To convert an array to a `Map`, use the `.toMap()` function.

Only an array of `Pair<K,V>` can be converted to a `Map`. The first value of a `Pair` instance becomes a key, and the second becomes a value. This example uses the infix notation to call the `to` function to create tuples of `Pair`:

```
val pairArray = arrayOf("apple" to 120, "banana" to 150, "cherry" to 90, "apple" to 140)

// Converts to a Map
// The keys are fruits and the values are their number of calories
// Note how keys must be unique, so the latest value of "apple"
// overwrites the first
println(pairArray.toMap())
// {apple=140, banana=150, cherry=90}
```

[Open in Playground →](#)

Target: JVM Running on v.2.4.0

Primitive-type arrays

If you use the `Array` class with primitive values, these values are boxed into objects. As an alternative, you can use primitive-type arrays, which allow you to store primitives in an array without the side effect of boxing overhead:

Primitive-type array	Equivalent in Java
<code>BooleanArray</code>	<code>boolean[]</code>
<code>ByteArray</code>	<code>byte[]</code>
<code>CharArray</code>	<code>char[]</code>
<code>DoubleArray</code>	<code>double[]</code>
<code>FloatArray</code>	<code>float[]</code>
<code>IntArray</code>	<code>int[]</code>
<code>LongArray</code>	<code>long[]</code>
<code>ShortArray</code>	<code>short[]</code>

These classes have no inheritance relation to the `Array` class, but they have the same set of

These classes have no inheritance relation to the `Array` class, but they have the same set of functions and properties.

This example creates an instance of the `IntArray` class:

+

```
// Creates an array of Int of size 5 with the values initialized to zero
val exampleArray = IntArray(5)
println(exampleArray.joinToString())
// 0, 0, 0, 0, 0
```

Open in Playground →

Target: JVM Running on v.24.0

i To convert primitive-type arrays to object-type arrays, use the `.toTypedArray()` ↗ function.

To convert object-type arrays to primitive-type arrays, use `.toBooleanArray()` ↗, `.toByteArray()` ↗, `.toCharArray()` ↗, and so on.

Any

Common JS JVM Native Wasm-JS Wasm-WASI

```
expect open class Any
```

(source)

The root of the Kotlin class hierarchy. Every Kotlin class has `Any` as a superclass.

Since Kotlin

1.0

Members Members & Extensions

Constructors

`Any`

Common JS JVM Native Wasm-JS Wasm-WASI

```
expect constructor()
```

Functions

`equals`

Common JS JVM Native Wasm-JS Wasm-WASI

```
expect open operator fun equals(other: Any?): Boolean
```

Indicates whether some other object is "equal to" this one.

Since Kotlin 1.0

`hashCode`

Common JS JVM Native Wasm-JS Wasm-WASI

```
expect open fun hashCode(): Int
```

Returns a hash code value for the object.

Since Kotlin 1.0

`toString`

Common JS JVM Native Wasm-JS Wasm-WASI

```
expect open fun toString(): String
```

Returns a string representation of the object.

Since Kotlin 1.0

Nothing

[Common](#) [JS](#) [JVM](#) [Native](#) [Wasm-JS](#) [Wasm-WASI](#)

```
expect class Nothing
```

[\(source\)](#)

Nothing has no instances. You can use Nothing to represent "a value that never exists": for example, if a function has the return type of Nothing, it means that it never returns (always throws an exception).

Since Kotlin

1.0

Unit

[Common](#) [JS](#) [JVM](#) [Native](#) [Wasm-JS](#) [Wasm-WASI](#)

```
expect object Unit
```

The type with only one value: the `Unit` object. This type corresponds to the `void` type in Java.

Since Kotlin

1.0

Members

Functions

[toString](#)

[JS](#) [JVM](#) [Native](#) [Wasm-JS](#) [Wasm-WASI](#)

```
open override fun toString(): String
```

Returns a string representation of the object.

Since Kotlin 1.1